

Comparative Performance Analysis of Javascript Frontend Web Frameworks

1st Jason Tong

Mathematic Department
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
Aboytong123@gmail.com

2nd Ricky Rivaldo Jikson

Mathematic Department
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
ricky.jikson@binus.ac.id

3rd Alexander Agung Santoso Gunawan

Computer Science Department
School of Computer Science
Bina Nusantara University
Jakarta, Indonesia 11480
aagung@binus.edu

Abstract— One of the most popular tools today for building engaging, robust, and easy to manage websites is the JavaScript programming language. Over the past 10 years, numerous front-end frameworks have been developed on top of JavaScript, making it simpler to develop, design, and maintain interactive websites. Currently, the most popular JavaScript front-end frameworks are React.js and Vue.js. Next.js is a relatively new framework that is also catching developers' attention and growing in popularity. This paper assesses the performance of these prominent front-end JavaScript frameworks, analyzing aspects like browser performance, speed index, interactive time, and the efficiency of specific DOM operations such as adding, editing, and removing elements. Using benchmark applications tailored for each framework, we conducted extensive testing to gauge their performance across different DOMs, leveraging technology-based data. Finally, our study of three front-end frameworks concluded that Next.js offers superior performance, React.js's focus on JavaScript dynamics leads to latency, and Vue.js exhibits average speed, particularly in 'to-do list' and 'image gallery' project contexts.

Keywords—JavaScript, Front-end Framework, React.js, Vue.js, Next.js.

I. INTRODUCTION

The purpose of this paper is to analyze performance of JavaScript front-end framework in determining how the developer can choose which framework that suits to their project or workplace.

Front-end Development is part of web development. At first, front-end development was just for displays content using HTML, CSS, and very minimum JavaScript even doesn't require a framework at all. To develop user interaction and effectiveness, developer need a new reusable and easy to maintain front-end technology [1]. At the same time, front-end developer has difficulties in using JavaScript programming languages because each type of browser has different support against standard ECMAScript. Framework is an enabling technology code structure to style quickly and effectively [2]. Most of the variations and complexities of JavaScript framework and library comes from front-end development itself. Front-end technology is developing very fast. several years then there is still using jQuery in manipulating elements, but React.js and Vue.js have started to take over the front-end market [3].

1) Performance Metrics for websites

For websites, performance metrics have a very important role. Good performance means less load time, higher satisfaction users, and can increase revenue for commercial websites.

Interaction speed is important performance metrics [4]. By reducing loading times by a few seconds, it can improve user interaction and storage. Speed interaction is also an important part when there are many requests from users and technical difficulties such as slow connection issues [5].

Due to complexity and variety of front-end frameworks, it will be so difficult for developers to select the right framework for their project, especially for beginners who are just start in front-end field [6]. Frameworks are also very sensitive because if framework changes then most of the code will change. Developers must choose based on the framework that is popular, frequently used, a decent framework as well as a front-end framework community. Not only that. In choosing a framework, you also have to consider high viability to ensure lots of maintenance and updates in the future [5]. The rest of this paper is organized as follows: first in this paper discuss about systematic literature review in general and then follow by a research methodology in section 3. In research methodology section, will discuss comparisons of JavaScript front-end developer and to find which is better or suitable for developer. Finally, we analyze our research results in section 4 and conclude our work with suggestions for future research in section 5.

II. TOOLS THAT WILL BE USED

To demonstrate and compare each framework, there will be some tools that will be used. It is important since the tools that has a relation to a browser.

A. Front-end Foundation (HTML and CSS)

HTML (Hyper Text Markup Language) is a markup language that is used to create and describe the structure and content of a web page. CSS (Cascading Style Sheets) is a language to visualize the web page which can be applied to HTML elements [7].

B. Javascript

1) Introduction to JavaScript

JavaScript is a scripting programming language that can be written in HTML. It was created in 1995 with the purpose to make interactive functions to web pages [8].

2) EcmaScript Standard

ECMAScript is a core language for JavaScript that performs computations in web applications. ECMAScript is standardized by Ecma International in ECMA-262 which specifies only language syntax and the semantics of the core API [4].

3) React.js

React.js is a framework library that can be used to make User Interfaces (UI). It has following helpful features, such as using virtual DOM to maximize efficiency, server-side rendering for some implementations, and can be customized [9].

4) *Vue.js*

Vue.js is a framework library that can be used as an addition to interactive behavior and functionality to any places that can be run by JavaScript. It can be used for simple tasks such as making individual webpages and complex tasks such as providing foundation for enterprise application [10].

5) *Next.js*

Next.js is a framework library that can be used to give React functionality's structure. It can also be used to identify and reload modified files, give advantage of using component-specific styles, and loads JavaScript modules and Reacts components [11].

III. METHODOLOGY

A. Experimental Methodology

For the next section, this paper will begin to analyze performance, user interaction, DOM and much more on each front-end framework. The goal is to help beginners to choosing the right framework for development. Therefore, this paper will build a Realworld web application using the different front-end frameworks to know the advantages and the disadvantage of the framework. finally, this paper will evaluate whether to validate each framework based on the results obtained [3].

B. Front-end Framework Selection

The survey team on JavaScript has invite tons of developers since 2016 to participate in a survey regarding the framework and libraries. From the survey, will get an idea about communities and ecosystems to know which front-end framework that developer must choose. Result of this survey was chosen in this paper because developer can know more about the most popular frameworks available currently based on ideas from thousands of developers around the world [3]. Survey in 2020 there were 23,000 more respondents from 137 countries, the survey covered ten types of frameworks or libraries.



Fig. 1. Satisfaction usage ratio rankings from State of JavaScript Surveys since 2016–2021.

From Fig. 1, React.js is front-end framework with the highest level of usability and followed by Angular and Vue.js in the 2021 usage rate ranking. React.js, Angular, and Vue.js are also quite stable since 2016 because it always in tops the top 3 until 2021. In addition to the framework above there are still many frameworks that don't make it into the top 10, one of them is Next.js. Next.js is a framework introduced in 2016. As technology develops, it becomes sufficient Good at

developing websites with attractive features offered and in 2021, it is still in the 13th rank [12].

The second data source to compare the front-end framework for this paper is npm daily front-end framework download [13]. Npm is an online repository for publishing of open-source Node.js projects. It has been used by more than 4 million developers on worldwide, and npm front-end framework download (Fig. 2) reflects the popularity of the project developed by developers.

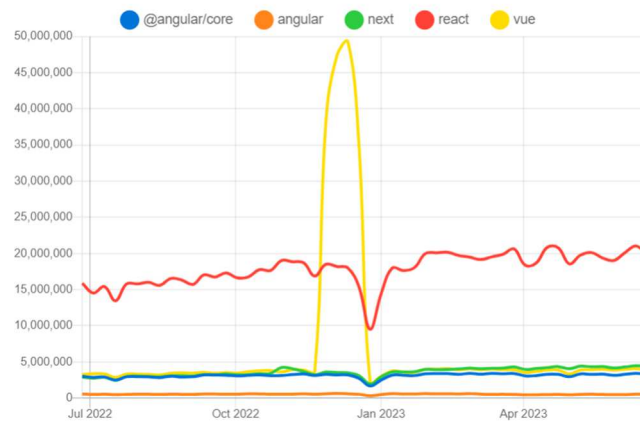


Fig. 2. Npm downloads of every frontend framework

The data was taken from December 2021 to December 2022. The number of Vue.js downloads is already over 30,000,000 and become first place between other frameworks. React.js download is rated second place with over 15,000,000 downloads about 1/2 that of Vue.js downloads. while other frameworks such as next.js and angular are still below the average is about 3,000,000 downloads this year [14].

C. Realworld Application

Realworld application (Fig. 3) is a case study of a web-based application which has the main features in doing CRUD (Create, Read, Update and Delete) To-do or usually called the Todolist App and features display photo gallery. In this Realworld application, it will be developed using 3 different frameworks namely React.js, Vue.js and Next.js. Each framework will be given the same data and use each method to produce the same result exactly.

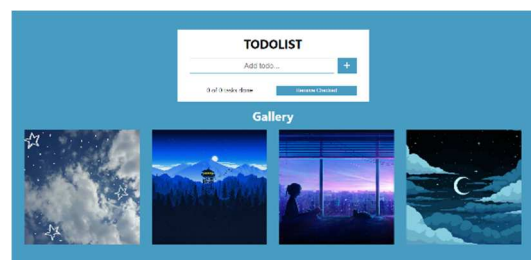


Fig. 3. Realworld application overview

1) *Building with React.js*

In general, in developing this Realworld application, React.js is not it's difficult to develop this app because react.js is basically a library with limited features so react.js requires third party libraries such as react.js-hooks for data management, as well as a library for content rendering to a DOM (document object model) that can be controlled effectively. In this section will discuss how to build the

Realworld Application using React.js. First, install create-react-app from npm or yarn and use create-react-app to generate a initial React.js project template in the terminal.

Main Page Component

```
import Todo from './Components/Todo/Todo';
```

Todolist Components

```
<Container>
  <h1>TODOLIST</h1>
  <InputContainer onSubmit={submitHandler}>
    <input type="text" ref={inputRef} />
    <button type="submit"></button>
  </InputContainer>
  <List todo={todo} setTodo={setTodo}></List>
  <Info>
    <p>
      {todo.filter((item) => item.status === true).length} of
      {todo.length}
      tasks done
    </p>
    <button onClick={RemoveChecked}> </button>
  </Info>
</Container>
```

Data management for storing Todo data in react.js will use one of the features of react.js hooks, namely useState. useState include 2 parameter variables, namely data value and set data value.

```
import React, {useState} from 'react.js';
let [todo, setTodo] = useState([]);
```

If user want to insert new todo, simply combine between ref variable and useState function to store the data. To store the data, program will automatically call function when user submitted.

```
let submitHandler = (e) => {
  e.preventDefault();
  if (!inputRef.current.value) return;

  setTodo((prev) => {
    return [
      ...prev,
      {text: inputRef.current.value, status: false, id: +new Date()},
    ];
  });
};
```

This function is used to store user input todo, todo status (start from false) and todo id in useState variable. Next to display todolist that user input all the time, simply use another component namely list. First, import list components to todolist components.

Todolist Component

```
import List from './List/List';
```

```
<Container>
  {todo.map((item, index) => (
    <TodoContainer key={index}>
      <input
        type="checkbox"
        id={index}
        onClick={() => checkToggle(index)}
      />
      <label for={index} className={item.status ? 'active' : null}>
        {item.text}
      </label>
    </TodoContainer>
  ))}
</Container>
```

Then, delete each todo by changing the status from false to true of the todo using the useState function that created earlier, namely setTodo. But, don't forget to also configure todo id that want to change.

```
let checkToggle = (id) => {
  let newTodo = [...todo];
  newTodo[id].status = !newTodo[id].status;
  setTodo(newTodo);
};
```

But in this function only change todo status and not actually delete the todo so simply create another function to delete all todo that have true status.

```
let RemoveChecked = () => {
  let newTodo = todo.filter((item) => item.status === false);
  setTodo(newTodo);
  document.querySelectorAll('input').forEach((item) =>
    (item.checked = false));
};
```

That's all for todolist feature and now time to create another feature namely image gallery. For image gallery, literally import image from local file and display using html img tag.

```
import image from './Assets/background.webp';
<img
  src={image}
  width={300}
  height={300}
  alt="wallpaper"
/>
```

This is an example for display image using react.js. But, if developer need more than 1 image, feel free to add more img tag with difference src value.

2) Building with Vue.js

Vue.js is different from React.js. Vue.js is equipped with features own data management so that it can be used without downloading other third-party libraries to develop this Realworld Application. First, install @vue.js/cli and create project using npm or yarn in the terminal. Same as react.js, vue.js also a component-based framework that can separate out to multiple smaller components, so in vue.js only need to separate our list file.

Main Component

```
<script>
import List from './components/ToDo.vue';
export default {
  components: { List }
};
</script>
```

List Component

```
<div class="Container">
  <div class="ToDoContainer" :key="index" v-for="(item, index) in
  todo">
    <input type="checkbox" :id="index" @click="setData(index)" />
    <label :for="index" :class="item.status ? 'active' : null">
      {{ item.text }}
    </label>
  </div>
</div>
```

Keep in mind that vue.js still using HTML based template so in vue.js can simply style inside the style tag.

```
<style scoped>
img {
  object-fit: cover;
}
.imgContainer {
  display: grid;
  grid-template-columns: repeat(auto-fit, 300px);
}
</style>
```

Also for data management don't need to import from external like react.js use Uestate. In vue.js can simply add "data" function in script export default object.

```
export default {
  data() {
    return {todo: [{id: 1, text: 'todolist - 1', status: false},{id: 2, text:
    'todolist - 2', status: false}]}
  }
}
```

And for several function to add todo to data management or delete todo, simply inset to "method" function in script export default object

```
export default {
  methods: {
    setData(index) {
      let todoList = [...this.todo]; todoList[index].status =
      !todoList[index].status; this.todo = todoList;
    },
    addTodo(event) {
      if (!event.target.querySelector('input').value) return;
      this.todo = [
        ...this.todo,
        { id: +new Date(),text: event.target.querySelector('input').value,
        status: false,
        },
      ];
    },
    RemoveChecked() {this.todo = this.todo.filter((item) => item.status
    === false);}
  },
}
```

To access our data management variables, simply use "this" keyword before variable name. For example, to change todo file simply write following code.

```
this.todo = this.todo.filter((item) => item.status === false);
```

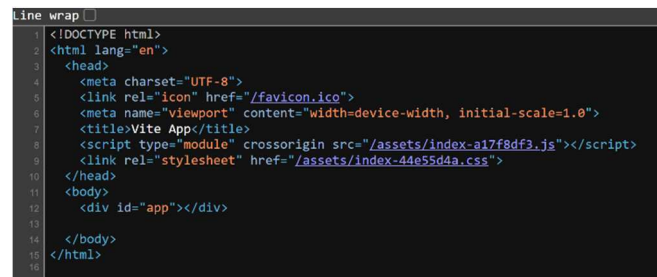
For add todo, remove todo, and other function for todolist feature use the same logic but a bit different syntax. To create image gallery in vue.js simply import image from local file inside script tag and store all image in our data management can use it in html template tag.

```
import image from './assets/background.webp';
export default {
  data() {return {image};}
}
```

3) Building with Next.js

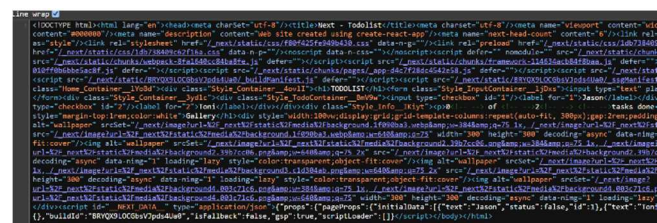
Building using next.js is literally the same while using React.js because next.js is an improvement technology from react.js library. So, in general, building Realworld applications is the same, but this section will explore next.js new feature that does not include in react.js library feature.

First, in next.js have maximize SEO (Search Engine Optimization) feature that vue.js and react.js cannot do that. To maximize SEO, use technology namely SSR (Server-Side Rendering) or SSG (Server-Side Generation). SSR is a program that will render our page before the browser displays it to client. So, while displaying it to client, our website is already having fully html tag in head and body tag that can boost our SEO value. The difference when using react.js and vue.js (Fig. 4) is although the result is same as next.js (Fig. 5), but UI displayed use react.js and vue.js framework is render in client side so in initial render website using react.js and vue.js framework do not have fully html tag in body tag and that is very impactful for SEO.



The screenshot shows the initial render code for a React.js application. It includes a full HTML document structure with a head section containing meta tags for charset, viewport, and title, and a body section with a single div element with the id 'app'.

Fig. 4. React.js and Vue.js initial render code



The screenshot shows the initial render code for a Next.js application. It displays a JSON object representing the props passed to the page component, including text, status, id, and name.

Fig. 5. Next.js initial render code

```
export const getStaticProps = async () => {
  return {
    props: {
      initData: [{text: 'Jason', status: false, id: 1},{text: 'Toni', status:
      false, id: 2}
    ]}
  };
}
```

In next.js, if developer want to create SSR simply create a function in main page namely “getStaticProps” and also need to export it as asynchronous function. Inside the function that return properties for initial render. In this situation only need to return our initial todo data to our data management (useState).

Second, in Next.js have a nice feature for display image in our website namely Image Compressor. Next.js can auto compress our image in website and change image type to Webp so image that have high size can way smaller and increase website performance. Here is how to create auto image compressor in Next.js.

```
import Image from 'next/image';
<Image
  src={image}
  width={300}
  height={300}
/>
```

To create auto image compressor in Next.js, first need to import special tag from Next.js Framework namely “Image”, then simply create Image tag and add properties like regular img tag from HTML. But in Image tag, must add 2 more properties namely “width” and “height” to determine height and width of the image.

IV. DATA ANALYSIS

In this section, a Realworld application that build with difference front end frameworks will differ in comparison based on performance, speed index and interactive time using lighthouse application that provide from chrome extension and manual analysis from website inspect [14]. This paper also provide insight about front-end framework and give a recommendations for developers to choose a framework that suitable for their development project.

A. Networking Realworld Application

1) Performance

When measuring performance on each framework, this paper will use lighthouse to measure it for 5 attempts and the result will be shown in Table 1 to Table 4.

TABLE I. AVERAGE PERFORMANCE USING FRONT-END FRAMEWORKS FOR REALWORLD APPLICATION USING LIGHTHOUSE

Test Attemp	Front-end Framework		
	React.js	Vue.js	Next.js
1	88	75	100
2	75	75	100
3	88	75	99
4	88	75	99
5	87	75	99
Average	85.2	75	99.4

If analyze the results of Lighthouse analysis, the conclude that next.js provides the best performance because of SSR and Image compressor, React.js is also very good with score of 85.2 and in the last position is vue.js which continues to be consistent with not high performance.

TABLE II. AVERAGE SPEED INDEX USING FRONT-END FRAMEWORKS FOR REALWORLD APPLICATION USING LIGHTHOUSE

Test Attemp	Front-end Framework		
	React.js	Vue.js	Next.js
1	0.9s	1.1s	0.8s
2	0.8s	1.1s	0.8s
3	0.8s	1.1s	0.8s
4	0.8s	1.1s	0.8s
5	0.8s	1.1s	0.8s
Average	0.82s	1.1s	0.8s

In lighthouse analysis, again and again can conclude that next.js provides the best speed index because of Image compressor feature which causes the image load time to be much faster, React.js is also very good with score of 0.82s and in the last position is vue.js which continues to be consistent with lowest speed index.

TABLE III. AVERAGE TIME TO INTERACT USING FRONT-END FRAMEWORKS FOR REALWORLD APPLICATION USING LIGHTHOUSE

Test Attemp	Front-end Framework		
	React.js	Vue.js	Next.js
1	1.5s	1.1s	1.5s
2	1.1s	1.2s	1.5s
3	1.1s	1.1s	1.2s
4	1.3s	1.1s	1.5s
5	1.3s	1.1s	1.5s
Average	1.26s	1.12s	1.44s

Time for interaction with the user is the total time required by the website to be fully interactive. this is a important part because some websites still prioritize visibility content over interactivity. This will be a big problem if website is completely loaded but when user tries to interact with our website and they don’t get any response. React.js and Vue.js are in moderate range, which is from 1.1 to 1.3s. Otherwise, Next.js are slower than React.js and Vue.js and the average time is 1.44s. React.js and Vue.js can provide users a pretty good interactive experience.

2) Line of Code

This paper will use the Cloc third library to calculate total lines of codes. Cloc can count total lines of code in src folder of each front-end framework without comment line and blank lines [15]. Total line of code can indication that how concise a given library/framework/language is.

TABLE IV. TOTAL LINE OF CODE FROM 3 DIFFERENT FRONT-END FRAMEWORK USING CLOC CODE STATISTIC TOOL

Front-end Framework	Total Line of Code
React.js	263
Vue.js	231
Next.js	269

Total lines of code indicates how concise the code is written for run realworld applications using a different front-end framework. This fact shows that Vue.js is a very well packaged and can provide same functionality with other front-end framework but fewer lines of code, whereas React.js and Next.js have a similar level of code simplicity.

3) Package Size

Packages with a smaller size can speed up the download time, the way to find out the size of the package is to look at the total size in the node_modules folder because all the code is listed in package.json stored in node_modules.

TABLE V. TOTAL PACKAGE SIZE FROM 3 DIFFERENT FRONT-END FRAMEWORK.

React.js	280mb
Vue.js	63.3mb
Next.js	148mb

Result of searching for package or package.json size to implement in realworld applications using 3 front-end libraries in the npm registry addressed are shown in Table 5, where the size obtained is dependencies or third party libraries that are used and registered inside package.json. Package size comparison the fact that React.js and Next.js support includes a greater spectrum of functionality, while Vue.js are created for much more efficient development that more suitable for small projects.

V. CONCLUSION

Through our exploration of three distinct front-end frameworks, we have gained a deeper understanding of their performance nuances, especially in the context of 'to-do list' and 'image gallery' project types. These projects serve as representations of user interactions and simple image rendering. Our study meticulously analyzed the associated DOM operations with CRUD (create, read, update, and delete) and the respective time frames for each framework. Our findings revealed that Next.js exhibits superior performance when compared to the other tested frameworks. React.js, on the other hand, does not achieve the speed of Next.js, primarily because it invests significant time in diverse DOM operations. This apparent latency in React.js can be attributed to its emphasis on optimizing JavaScript dynamics over accelerating the Diff Virtual DOM. Meanwhile, Vue.js demonstrated average speed performance.

After conducting this research, it became evident that each front-end framework brings its unique advantages and characteristics, catering to developers' varied needs in website projects. However, this study possesses certain limitations. The 'Realworld' application, though insightful, represents a relatively simple web project, and the performance of these frameworks may differ with more complex applications. For the future research, we can augment this study by introducing more intricate web projects and including other emerging front-end frameworks. This would provide a broader understanding of their capabilities and performance nuances.

REFERENCES

- [1] D. A. Rauschmayer, *Speaking JavaScript: An In-Depth Guide for Programmers*, California: O'Reilly Media, 2014.
- [2] C. Fu, "Exploration of Web front-end development technology and optimization," Atlantis Press, pp. 166-169, 2016.
- [3] M. Kaluža, K. Troskot and B. Vukelić, "COMPARISON OF FRONT-END FRAMEWORKS FOR," *Zbornik Veleučilišta u Rijeci*, vol. 6, no. 1, pp. 261-282, 2018.
- [4] D.-P. P. a. A. Altar, "Designing an MVC Model for Rapid Web Application," *Procedia Engineering*, vol. 69, pp. 1172-1179, 2014.
- [5] R. Vyas, "Comparative Analysis on Front-End Frameworks," *International Journal for Research in Applied Science & Engineering Technology*, vol. 10, no. VII, pp. 298-307, 2022.
- [6] E. Molin, "Comparison of Single-Page Application Frameworks : A method of," *KTH ROYAL INSTITUTE OF TECHNOLOGY*, Sweden, 2016.
- [7] D. Raggett, A. L. Hors and I. Jacobs, *HTML 4.0 Specification*, Cambridge: w3c, 1998.
- [8] L. Groner, *Learning JavaScript Data Structures and Algorithms: Write complex and powerful JavaScript code using the latest ECMAScript*, 3rd Edition, Birmingham: Packt Publishing Ltd, 2018.
- [9] Z. Dinku, "React.js vs. Next.js," *Metropolia*, Finland, 2022.
- [10] E. Hanchett and h. Listwon, *Vue.js in Action*, New york: Simon and Schuster, 2018.
- [11] Y. Ko, H. Lee, J. Dolby and S. Ryu, "Practically Tunable Static Analysis Framework for Large-Scale JavaScript Applications (T)," in *30th IEEE/ACM International Conference on Automated Software Engineering*, 2015, 2015.
- [12] Netlify, "2021stateofjs," Netlify, 2021. [Online]. Available: <https://2021.stateofjs.com/en-US/libraries/front-end-frameworks/>. [Accessed 28 december 2022].
- [13] J. Potter, "npmtrends," uidotdev, [Online]. Available: <https://npmtrends.com/>. [Accessed 28 december 2022].
- [14] G. Developers, "Tools for Web Developers," Lighthouse, [Online]. Available: <https://developer.chrome.com/docs/lighthouse/overview/>. [Accessed 2 january 2023].
- [15] A. Danial, "cloc," Github, August 2006. [Online]. Available: <https://github.com/AlDanial/cloc>. [Accessed 3 January 2023]